

P2406R5

Add ``lazy_counted_iterator``

Published Proposal, 2023-02-08

This version:

<https://wg21.link/P2406>

Authors:

[Yehezkel Bernat](#)

[Yehuda Bernat](#)

Audience:

SG9 (RANGES), LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

[GitHub](#)

Abstract

``counted_iterator`` increments its internal iterator even when reaching its own end, which makes it unusable in some cases, especially for input iterators. This paper suggests adding ``lazy_counted_iterator`` alternative to be used in such cases

Table of Contents

1	Revision History
2	Problem description
2.1	Range with the exact number of items
2.2	<code>input_iterator</code> case
3	Current behavior is what the standard mandates
4	Desired behavior
5	High-level design of the proposed solution
6	Design points for discussion
6.1	Constructing with 0 count
6.2	Return type of <code>operator++(int)</code>
6.3	Why <code>lazy_take</code> instead of fixing <code>take</code> ?
7	Design alternative
7.1	Motivation
7.1.1	Poll outcomes
7.1.1.1	POLL: Reject C++23 National Body comment [US-46-107]
7.1.1.2	POLL: Forward P2406R4 to LWG for C++ 23
7.1.1.3	POLL: Change <code>counted_iterator</code> to have the proposed behavior of <code>lazy_counted_iterator</code> , which is a breaking change.
7.1.1.4	POLL: Change iterator category to at most <code>forward</code>
7.1.1.5	POLL: Require <code>forward</code> as a minimum underlying iterator category
7.1.1.6	POLL: Remove <code>base</code> from <code>lazy_counted_iterator</code>
7.2	Alternative 1: Replace <code>counted_iterator</code> with <code>lazy_counted_iterator</code>
7.3	Alternative 2: Add <code>lazy_counted_iterator</code> and deprecate <code>counted_iterator</code>
7.3.1	Alternative 2 variation II: Partial deprecation
8	Wording
8.1	Wording for <code>lazy_counted_iterator</code>
8.1.1	x.1 Class template <code>lazy_counted_iterator</code> [lazy.counted.iterator]

- 8.1.2 x.2 Constructors and conversions [lazy.counted.iter.const]
- 8.1.3 x.3 Accessors [lazy.counted.iter.access]
- 8.1.4 x.4 Element access [lazy.counted.iter.elem]
- 8.1.5 x.5 Navigation [lazy.counted.iter.nav]
- 8.1.6 x.6 Comparisons [lazy.counted.iter.cmp]
- 8.1.7 x.7 Customizations [lazy.counted.iter.cust]
- 8.2 Wording for `views::lazy_counted` and `lazy_take_view`
- 8.3 Wording for `views::lazy_counted`
 - 8.3.1 26.7.x Lazy counted view [range.lazy.counted]
- 8.4 Wording for `lazy_take_view`
 - 8.4.1 x.1 Overview [range.lazy.take.overview]
 - 8.4.2 x.2 Class template `lazy_take_view` [range.lazy.take.view]
 - 8.4.3 x.3 Class template `lazy_take_view::sentinel` [range.lazy.take.sentinel]
- 8.5 Feature test macro

9 Opens

10 Implementation experience

11 Note about optimization

12 Acknowledgements

References

Informative References

§ 1. Revision History

r5: Updates following LEWG review (Issaquah - 2023-02-07 morning)

- Incorporate poll results
- Suggest design alternatives
- Add missing feature test macro

r4: Integrate SG9 feedback (Issaquah)

- Fix missing change from `input_or_output_iterator` to `input_iterator` in the synopsis

- Simplify the definition of when two iterators refer to the same sequence
- Simplify the definition of `void operator++(int)`
- Add [opens](#) section for additional design suggestions/questions raised
- Add [implementation experience](#) section

r3: Integrating LEWG feedback:

- Define `iterator_concept` and `iterator_category` like other iterator adaptors do, unlike `counted_iterator` which is special
- Require `input_iterator` as `operator++(int)` definition doesn't match `output_iterator` requirements
- General cleanup

r2: Integrating SG9 feedback:

- Removing references to p2578, after SG9 vote against it
- Fix design suggested
- Add design alternatives

r1: Improving many parts, following feedback from Inbal Levi and from Reddit users

r0: initial revision

§ 2. Problem description

§ 2.1. Range with the exact number of items

Look at this example code [\[CE-FILTER\]](#):

```
#include <ranges>
#include <iostream>

namespace rv = std::views;

int main() {
    for (auto i : rv::iota(0)
        | rv::filter([](auto i) { return i < 11; }))
```

```

        | rv::take(11))
    std::cout << i << '\n';
}

```

Compiler explorer gets a timeout when trying to run this simple example, instead of printing the numbers from 0 to 10. Running the same code locally, it runs for very long time. Tracking the roots of the issue, the problem is that `take` uses `counted_iterator` when the range isn't `random_access` and `counted_iterator` increments the internal iterator even if the counter has reached the requested count. In this case, the filter never returns when trying to increment it once again (at least not until `iota` reaches the UB case of signed overflow).

The example above is just for illustration, but we can think about cases where it isn't clear for the user how many items the filter is expected to return, so limiting the output count with `take` becomes dangerous and results in unexpected behavior.

It means `take` isn't usable on ranges if we don't know in advance that there is an extra element in the range.

§ 2.2. `input_iterator` case

Even more common problem is when using input ranges, e.g. `basic_istream_view`. In most of these cases, advancing the internal iterator when reaching the count means eating an additional input that can't be retrieved again later, or hanging forever if no additional input exists and the stream isn't closed. For example [\[CE-ISTREAM\]](#):

```

#include <ranges>
#include <iostream>
#include <sstream>
#include <cassert>

namespace rn = std::ranges;
namespace rv = rn::views;

int main()
{
    auto iss = std::istringstream("0 1 2");
    for (auto i : rn::istream_view<int>(iss)
        | rv::take(1))
        std::cout << i << '\n';
    auto i = 0;
}

```

```
iss >> i;
std::cout << i << std::endl; // flush it in case the assert fails
assert(i == 1); // FAILS, i == 2
}
```

It makes it harder to use ranges for things like parsing input, if the rest of the stream is still to be used or we aren't sure there is any additional element in the stream.

Seems like this was discussed in [\[range-v3-issue57\]](#), and there was no decision what is the right solution.

§ 3. Current behavior is what the standard mandates

Under 23.5.6.5 [counted.iter.nav], the standard defines the behavior of `operator++` for `counted_iterator` as:

Effects: Equivalent to:

```
++current;
--length;
return *this;
```

It means that even when `length` becomes 0, the internal iterator is incremented, thus consuming an additional item from the range, and causing the effects mentioned above for input iterator case or when `++` on the internal iterator is costly (or never returns).

§ 4. Desired behavior

As long as `counted_iterator` is valid (not equal to `default_sentinel`), it must never try to access more than `n` items (when `n` is the given count). If the range doesn't have `n` items, the behavior is kept as is, i.e. it isn't defined (`operator++` might hang forever or access things that shouldn't be accessed etc.).

§ 5. High-level design of the proposed solution

We propose adding a new iterator type, `lazy_counted_iterator`. This type behaves similarly to `counted_iterator`, with changes to its operator definition around 0 count so it doesn't increment the internal iterator when reaching 0 count.

Additionally, this requires adding `lazy_take` and `views::lazy_counted` that uses the new iterator instead of `counted_iterator`.

§ 6. Design points for discussion

§ 6.1. Constructing with 0 count

Similarly to `counted_iterator`, `lazy_counted_iterator` must allow constructing with 0 count. In most design alternatives, this puts the iterator in an inconsistent internal state, as the underlying iterator is expected to be "one step back".

Please note that `base()` and decrementing are the only operations involving the state of the internal iterator and still legal for `counted_iterator` constructed with `n==0`.

The solution accepted in SG9 is to:

1. Cap `lazy_counted_iterator` to `forward_iterator`, so decrementing is never supported.
2. Don't provide `base()` method, so there is no way to observe the inconsistency and get unexpected behavior in some cases.

This also simplifies the implementation, as there is no requirement to differentiate between these two states of the underlying iterator. (Implementations might still decide to track it for providing additional diagnostics for violations of the precondition of iterator comparison.)

§ 6.2. Return type of `operator++(int)`

For non-forward iterators, today `counted_iterator::operator++(int)` is defined with `return current++;` and `decltype(auto)`, as such an iterator might return a different type or not return anything at all (e.g. if it's move only iterator). `input_or_output_iterator` is `weakly_incrementable`, not `incrementable`. As we don't always increment the iterator, there is no consistent type to return. As a result, for non-forward iterators, we define `operator++(int)` as returning `void`. This also prevents us from supporting `output_iterator`, as one of its requirements is to support `*it++`.

§ 6.3. Why `lazy_take` instead of fixing `take`?

We could have change `take` to use `lazy_counted_iterator` when constructed with input (non lazy) range. Besides ABI considerations, we find it wrong if `take` used to return one type (`counted_iterator`) and now will start returning a different one, `lazy_counted_iterator`, as this is source-breaking change. Additionally, as demonstrated above, there are cases where the user wants using `lazy_counted_iterator` on forward iterators too, but this is something that only the user know and we can't automatically detect and decide on behalf of them. We can't change all cases of `take` to use `lazy_counted_iterator`, due to the differences in behavior both for lazy input iterators and forward iterators (that are not random access), as described below.

We aren't happy with the additional burden on teachability, but we believe in most cases users can just use `lazy_take` and it does The Right Thing. The only point where users must be aware of it is when they use `base()` method, which we expect to be quite advance usage in general. Users who care about absolute performance, can choose using `take` when they know it works correctly for their case.

§ 7. Design alternative

§ 7.1. Motivation

LEWG reviewed R4 of this paper (Issaquah - 2023-02-07 morning) with the following poll results:

§ 7.1.1. Poll outcomes

§ 7.1.1.1. POLL: Reject C++23 National Body comment [\[US-46-107\]](#)

SF	WF	N	WA	SA
3	8	9	2	2

Outcome: No consensus

WA: Preference to break C++20 sooner

§ 7.1.1.2. POLL: Forward P2406R4 to LWG for C++ 23

SF	WF	N	WA	SA
1	0	5	12	5

Outcome: Consensus against.

- § 7.1.1.3. POLL: Change `counted_iterator` to have the proposed behavior of `lazy_counted_iterator`, which is a breaking change.

SF	WF	N	WA	SA
6	12	2	1	3

Outcome: Consensus in favor.

- § 7.1.1.4. POLL: Change iterator category to at most `forward`

SF	WF	N	WA	SA
0	8	7	0	2

Outcome: No consensus, but leaning towards favoring this.

- § 7.1.1.5. POLL: Require `forward` as a minimum underlying iterator category

SF	WF	N	WA	SA
0	3	5	3	3

Outcome: No consensus.

- § 7.1.1.6. POLL: Remove `base` from `lazy_counted_iterator`

SF	WF	N	WA	SA
1	9	1	3	1

Outcome: Weak consensus in favor.

- § 7.2. Alternative 1: Replace `counted_iterator` with `lazy_counted_iterator`

This alternative propose to change `counted_iterator` to be what this paper suggests as `lazy_counted_iterator` or a variation of it.

Issues with this alternative are:

1. This is breaking change (both ABI and source breaking, and depending on the design it might be a silent break of behavior change too)
2. At least one implementer can't/won't ship it. Another implementer could ship it but seems to vaguely prefer not to and is wondering whether or not breaking this now is worth it
3. It would require that we lose `base()` (which LEWG opposed) since otherwise there'd be a behavior-breaking/dangerous change

§ 7.3. Alternative 2: Add `lazy_counted_iterator` and deprecate `counted_iterator`

This alternative strives to reduce to additional complexity in teachability and direct the users to the right direction.

The proposal here is to adopt `lazy_counted_iterator`, `views::lazy_counted` and `lazy_take` as proposed in this paper and mark `counted_iterator`, `views::counted` and `take` as deprecated. We could either specify attribute `[[deprecated]]`, or, to allow greater implementer freedom, deprecation here could mean adding a non-normative note that the compiler should warn in these cases.

§ 7.3.1. Alternative 2 variation II: Partial deprecation

To allow things like `vector | take(n)` to keep being `random_access_iterator`, we want to suggest here to deprecate only the cases of `counted_iterator` used `input_iterator` or `forward_iterator`, keeping the cases of `output_iterator` and `bidirectional_iterator` (and better) as they are.

We also want LEWG to consider the case of `istreambuf_iterator` (or any iterator considered as "lazy input iterator", see [\[D2578R0\]](#) for further discussion), where current behavior of `take` does the right thing. Forcing the user to use `lazy_take` and then incrementing the iterator once again to continue reading from the right location seems like another teachability issue.

§ 8. Wording

§ 8.1. Wording for `lazy_counted_iterator`

Under Header `<iterator>` synopsis `[iterator.syn]` add the new type:

```
// [iterators.counted], counted iterators
template<input_or_output_iterator I> class counted_iterator;

template<input_iterator I>
    requires see below
    struct iterator_traits<counted_iterator<I>>;           // freestanding
```

```
// [iterators.lazy.counted], lazy counted iterators
template<input_iterator I> class lazy_counted_iterator;           // fre
```

In Iterator adaptors `[predef.iterators]`, after 25.5.7 Counted iterators `[iterators.counted]` add new section:

25.5.x Lazy counted iterators `[iterators.lazy.counted]`

Under this section add:

§ 8.1.1. x.1 Class template `lazy_counted_iterator` `[lazy.counted.iterator]`

Class template `lazy_counted_iterator` is an iterator adaptor with the same behavior as the underlying iterator except that it keeps track of the distance to the end of its range. It can be used together with `default_sentinel` in calls to generic algorithms to operate on a range of N elements starting at a given position without needing to know the end position a priori.

[Example 1:

```
list<string> s;
// populate the list s with at least 10 strings
vector<string> v;
// copies 10 strings into v:
ranges::copy(lazy_counted_iterator(s.begin(), 10), default_sentinel, back_i
```

— end example]

Two values `i1` and `i2` of types `lazy_counted_iterator<I1>` and `lazy_counted_iterator<I2>` refer to elements of the same sequence if and only if there exists some integer `n` such that `next(i1.current, i1.count() + n)` and `next(i2.current, i2.count() + n)` refer to the same (possibly past-the-end) element.

```
namespace std {
    template<input_iterator I>
    class lazy_counted_iterator {
    public:
        using iterator_type = I;
        using value_type = iter_value_t<I>;
        using difference_type = iter_difference_t<I>;
        using iterator_concept = see below;
        using iterator_category = see below; // not always
        constexpr lazy_counted_iterator() requires default_initializable<I> = c
        constexpr lazy_counted_iterator(I x, iter_difference_t<I> n);
        template<class I2>
            requires convertible_to<const I2&, I>
            constexpr lazy_counted_iterator(const lazy_counted_iterator<I2>& x)

        template<class I2>
            requires assignable_from<I&, const I2&>
            constexpr lazy_counted_iterator& operator=(const lazy_counted_itera

        constexpr iter_difference_t<I> count() const noexcept;
        constexpr decltype(auto) operator*();
        constexpr decltype(auto) operator*() const
            requires dereferenceable<const I>;

        constexpr lazy_counted_iterator& operator++();
        constexpr void operator++(int);
        constexpr lazy_counted_iterator operator++(int)
            requires forward_iterator<I>;

        template<common_with<I> I2>
            friend constexpr iter_difference_t<I2> operator-(
                const lazy_counted_iterator& x, const lazy_counted_iterator<I2>& y)
            friend constexpr iter_difference_t<I> operator-(
                const lazy_counted_iterator& x, default_sentinel_t);
            friend constexpr iter_difference_t<I> operator-(
```

```

    default_sentinel_t, const lazy_counted_iterator& y);

template<common_with<I> I2>
    friend constexpr bool operator==(
        const lazy_counted_iterator& x, const lazy_counted_iterator<I2>& y)
    friend constexpr bool operator==(
        const lazy_counted_iterator& x, default_sentinel_t);

template<common_with<I> I2>
    friend constexpr strong_ordering operator<=> (
        const lazy_counted_iterator& x, const lazy_counted_iterator<I2>& y)

    friend constexpr iter_rvalue_reference_t<I> iter_move(const lazy_counted_iterator& i)
        noexcept(noexcept(ranges::iter_move(i.current)));
template<indirectly_swappable<I> I2>
    friend constexpr void iter_swap(const lazy_counted_iterator& x, const lazy_counted_iterator<I2>& y)
        noexcept(noexcept(ranges::iter_swap(x.current, y.current)));

private:
    I current = I(); // exposition only
    iter_difference_t<I> length = 0; // exposition only
};
}

```

The member typedef-name `iterator_concept` denotes

- `forward_iterator_tag` if `Iterator` models `forward_iterator`, and
- `input_iterator_tag` otherwise.

The member typedef-name `iterator_category` is defined if and only if the qualified-id `iterator_traits<Iterator>::iterator_category` is valid and denotes a type. In that case, `iterator_category` denotes

- `forward_iterator_tag` if the type `iterator_traits<Iterator>::iterator_category` models `derived_from<forward_iterator_tag>`, and
- `iterator_traits<Iterator>::iterator_category` otherwise.

§ 8.1.2. x.2 Constructors and conversions [lazy.counted.iter.const]

```
constexpr lazy_counted_iterator(I i, iter_difference_t<I> n);
```

Preconditions: `n >= 0`.

Effects: Initializes `current` with `std::move(i)` and `length` with `n`.

```
template<class I2>
    requires convertible_to<const I2&, I>
    constexpr lazy_counted_iterator(const lazy_counted_iterator<I2>& x);
```

Effects: Initializes `current` with `x.current` and `length` with `x.length`.

```
template<class I2>
    requires assignable_from<I&, const I2&>
    constexpr lazy_counted_iterator& operator=(const lazy_counted_iterator<
```

Effects: Assigns `x.current` to `current` and `x.length` to `length`.

Returns: `*this`.

§ 8.1.3. x.3 Accessors [lazy.counted.iter.access]

```
constexpr iter_difference_t<I> count() const noexcept;
```

Effects: Equivalent to: `return length;`

§ 8.1.4. x.4 Element access [lazy.counted.iter.elem]

```
constexpr decltype(auto) operator*();
constexpr decltype(auto) operator*() const
    requires dereferenceable<const I>;
```

Preconditions: `length > 0` is true.

Effects: Equivalent to: `return *current;`

§ 8.1.5. x.5 Navigation [lazy.counted.iter.nav]

```
constexpr lazy_counted_iterator& operator++();
```

Preconditions: `length > 0`.

Effects: Equivalent to:

```
if (length > 1) ++current;
--length;
return *this;
```

```
constexpr void operator++(int);
```

Preconditions: `length > 0`.

Effects: Equivalent to:

```
++*this;
```

```
constexpr lazy_counted_iterator operator++(int)
    requires forward_iterator<I>;
```

Effects: Equivalent to:

```
lazy_counted_iterator tmp = *this;
++*this;
return tmp;
```

```
template<common_with<I> I2>
    friend constexpr iter_difference_t<I2> operator-(
        const lazy_counted_iterator& x, const lazy_counted_iterator<I2>& y);
```

Preconditions: `x` and `y` refer to elements of the same sequence ([lazy.counted.iterator]).

Effects: Equivalent to: `return y.length - x.length;`

```
friend constexpr iter_difference_t<I> operator-(
    const lazy_counted_iterator& x, default_sentinel_t);
```

Effects: Equivalent to: `return -x.length;`

```
friend constexpr iter_difference_t<I> operator-(
    default_sentinel_t, const lazy_counted_iterator& y);
```

Effects: Equivalent to: `return y.length;`

§ 8.1.6. x.6 Comparisons [lazy.counted.iter.cmp]

```
template<common_with<I> I2>
    friend constexpr bool operator==(
        const lazy_counted_iterator& x, const lazy_counted_iterator<I2>& y);
```

Preconditions: `x` and `y` refer to elements of the same sequence ([lazy.counted.iterator]).

Effects: Equivalent to: `return x.length == y.length;`

```
friend constexpr bool operator==(
    const lazy_counted_iterator& x, default_sentinel_t);
```

Effects: Equivalent to: `return x.length == 0;`

```
template<common_with<I> I2>
    friend constexpr strong_ordering operator<==(
        const lazy_counted_iterator& x, const lazy_counted_iterator<I2>& y);
```

Preconditions: `x` and `y` refer to elements of the same sequence ([lazy.counted.iterator]).

Effects: Equivalent to: `return y.length <== x.length;`

[Note 1: The argument order in the *Effects:* element is reversed because `length` counts down, not up.
— end note]

§ 8.1.7. x.7 Customizations [lazy.counted.iter.cust]

```
friend constexpr iter_rvalue_reference_t<I>
    iter_move(const lazy_counted_iterator& i)
        noexcept(noexcept(ranges::iter_move(i.current)));
```

Preconditions: `i.length > 0` is true.

Effects: Equivalent to: `return ranges::iter_move(i.current);`

```
template<indirectly_swappable<I> I2>
    friend constexpr void
        iter_swap(const lazy_counted_iterator& x, const lazy_counted_iterator<I2>& y) noexcept(
            noexcept(ranges::iter_swap(x.current, y.current)));
```

Preconditions: Both `x.length > 0` and `y.length > 0` are true.

Effects: Equivalent to `ranges::iter_swap(x.current, y.current)`.

§ 8.2. Wording for `views::lazy_counted` and `lazy_take_view`

Under Header `<ranges>` synopsis **[ranges.syn]** add the new types:

```
// [range.counted], counted view
namespace views { inline constexpr unspecified counted = unspecified; }
```

```
// [range.lazy.counted], lazy counted view
namespace views { inline constexpr unspecified lazy_counted = unspecified; }
```

```
// [range.take], take view
template<view> class take_view;           // freestanding

template<class T>
    constexpr bool enable_borrowed_range<take_view<T>> =           // freestanding
        enable_borrowed_range<T>;

namespace views { inline constexpr unspecified take = unspecified; }
```

```
// [range.lazy.take], lazy take view
template<view> class lazy_take_view;     // freestanding

template<class T>
```

```
constexpr bool enable_borrowed_range<lazy_take_view<T>> = // freesta
    enable_borrowed_range<T>;

namespace views { inline constexpr unspecified lazy_take = unspecified; }
```

§ 8.3. Wording for `views::lazy_counted`

In Range adaptors [`range.adaptors`], after 26.7.18 Counted view [`range.counted`] add new section:

§ 8.3.1. 26.7.x Lazy counted view [`range.lazy.counted`]

A lazy counted view presents a view of the elements of the counted range
 ([iterator.requirements.general]) `i + [0, n)` for an iterator `i` and non-negative integer `n`.

The name `views::lazy_counted` denotes a customization point object
 ([customization.point.object]). Let `E` and `F` be expressions, let `T` be `decay_t<decltype((E))>`,
 and let `D` be `iter_difference_t<T>`. If `decltype((F))` does not model `convertible_to<D>`,
`views::lazy_counted(E, F)` is ill-formed.

[Note 1: This case can result in substitution failure when `views::lazy_counted(E, F)` appears in
 the immediate context of a template instantiation. — end note]

Otherwise, `views::lazy_counted(E, F)` is expression-equivalent to:

- If `T` models `contiguous_iterator`, then `span(to_address(E), static_cast<size_t>(static_cast<D>(F)))`.
- Otherwise, if `T` models `random_access_iterator`, then `subrange(E, E + static_cast<D>(F))`, except that `E` is evaluated only once.
- Otherwise, `subrange(lazy_counted_iterator(E, F), default_sentinel)`.

§ 8.4. Wording for `lazy_take_view`

After 26.7.10 Take view [`range.take`] add new section:

26.7.x Lazy take view [`range.lazy.take`]

Under this section add:

§ 8.4.1. x.1 Overview [range.lazy.take.overview]

`lazy_take_view` produces a view of the first N elements from another view, or all the elements if the adapted view contains fewer than N .

The name `views::lazy_take` denotes a range adaptor object (`[range.adaptor.object]`). Let E and F be expressions, let T be `remove_cvref_t<decltype((E))>`, and let D be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model `convertible_to<D>`, `views::lazy_take(E, F)` is ill-formed. Otherwise, the expression `views::lazy_take(E, F)` is expression-equivalent to:

- If T is a specialization of `ranges::empty_view` (`[range.empty.view]`), then `((void)F, decay-copy(E))`, except that the evaluations of E and F are indeterminately sequenced.
- Otherwise, if T models `random_access_range` and `sized_range` and is a specialization of `span` (`[views.span]`), `basic_string_view` (`[string.view]`), or `ranges::subrange` (`[range.subrange]`), then `U(ranges::begin(E), ranges::begin(E) + std::min<D>(ranges::distance(E), F))`, except that E is evaluated only once, where U is a type determined as follows:
 - if T is a specialization of `span`, then U is `span<typename T::element_type>`;
 - otherwise, if T is a specialization of `basic_string_view`, then U is T ;
 - otherwise, T is a specialization of `ranges::subrange`, and U is `ranges::subrange<iterator_t<T>>`;
 - otherwise, if T is a specialization of `ranges::iota_view` (`[range.iota.view]`) that models `random_access_range` and `sized_range`, then `ranges::iota_view(*ranges::begin(E), *(ranges::begin(E) + std::min<D>(ranges::distance(E), F)))`, except that E is evaluated only once.
- Otherwise, if T is a specialization of `ranges::repeat_view` (`[range.repeat.view]`):
 - if T models `sized_range`, then `views::repeat(*E.value_, std::min<D>(ranges::distance(E), F))` except that E is evaluated only once;
 - otherwise, `views::repeat(*E.value_, static_cast<D>(F))`.
- Otherwise, `ranges::lazy_take_view(E, F)`.

[Example 1:

```
vector<int> is{0,1,2,3,4,5,6,7,8,9};
for (int i : is | views::lazy_take(5))
```

```
cout << i << ' '; // prints 0 1 2 3 4
```

— end example]

§ 8.4.2. x.2 Class template `lazy_take_view` [range.lazy.take.view]

```
namespace std::ranges {
    template<view V>
    class lazy_take_view : public view_interface<lazy_take_view<V>> {
    private:
        V base_ = V(); // exposition only
        range_difference_t<V> count_ = 0; // exposition only

        // [range.lazy.take.sentinel], class template lazy_take_view::sentinel
        template<bool> class sentinel; // exposition only

    public:
        lazy_take_view() requires default_initializable<V> = default;
        constexpr lazy_take_view(V base, range_difference_t<V> count);

        constexpr V base() const & requires copy_constructible<V> { return base; }
        constexpr V base() && { return std::move(base_); }

        constexpr auto begin() requires (!simple-view<V>) {
            if constexpr (sized_range<V>) {
                if constexpr (random_access_range<V>) {
                    return ranges::begin(base_);
                } else {
                    auto sz = range_difference_t<V>(size());
                    return lazy_counted_iterator(ranges::begin(base_), sz);
                }
            } else if constexpr (sized_sentinel_for<sentinel_t<V>, iterator_t<V>>) {
                auto it = ranges::begin(base_);
                auto sz = std::min(count_, ranges::end(base_) - it);
                return lazy_counted_iterator(std::move(it), sz);
            } else {
                return lazy_counted_iterator(ranges::begin(base_), count_);
            }
        }

        constexpr auto begin() const requires range<const V> {
```

```

if constexpr (sized_range<const V>) {
    if constexpr (random_access_range<const V>) {
        return ranges::begin(base_);
    } else {
        auto sz = range_difference_t<const V>(size());
        return lazy_counted_iterator(ranges::begin(base_), sz);
    }
} else if constexpr (sized_sentinel_for<sentinel_t<const V>, iterator_t<const V>>) {
    auto it = ranges::begin(base_);
    auto sz = std::min(count_, ranges::end(base_) - it);
    return lazy_counted_iterator(std::move(it), sz);
} else {
    return lazy_counted_iterator(ranges::begin(base_), count_);
}
}

constexpr auto end() requires (!simple_view<V>) {
    if constexpr (sized_range<V>) {
        if constexpr (random_access_range<V>)
            return ranges::begin(base_) + range_difference_t<V>(size());
        else
            return default_sentinel;
    } else if constexpr (sized_sentinel_for<sentinel_t<V>, iterator_t<V>>)
        return default_sentinel;
    } else {
        return sentinel<false>{ranges::end(base_)};
    }
}

constexpr auto end() const requires range<const V> {
    if constexpr (sized_range<const V>) {
        if constexpr (random_access_range<const V>)
            return ranges::begin(base_) + range_difference_t<const V>(size());
        else
            return default_sentinel;
    } else if constexpr (sized_sentinel_for<sentinel_t<const V>, iterator_t<const V>>)
        return default_sentinel;
    } else {
        return sentinel<true>{ranges::end(base_)};
    }
}

constexpr auto size() requires sized_range<V> {

```

```

    auto n = ranges::size(base_);
    return ranges::min(n, static_cast<decltype(n)>(count_));
}

constexpr auto size() const requires sized_range<const V> {
    auto n = ranges::size(base_);
    return ranges::min(n, static_cast<decltype(n)>(count_));
}
};

template<class R>
    lazy_take_view(R&&, range_difference_t<R>)
        -> lazy_take_view<views::all_t<R>>;
}

```

`constexpr lazy_take_view(V base, range_difference_t<V> count);`

Preconditions: `count >= 0` is true.

Effects: Initializes `base_` with `std::move(base)` and `count_` with `count`.

§ 8.4.3. x.3 Class template `lazy_take_view::sentinel` [range.lazy.take.sentinel]

```

namespace std::ranges {
    template<view V>
    template<bool Const>
    class lazy_take_view<V>::sentinel {
    private:
        using Base = maybe_const<Const, V>;
        template<bool OtherConst>
            using CI = lazy_counted_iterator<iterator_t<maybe_const<OtherConst, V>,
            sentinel_t<Base> end_ = sentinel_t<Base>();

    public:
        sentinel() = default;
        constexpr explicit sentinel(sentinel_t<Base> end);
        constexpr sentinel(sentinel_t<Const> s)
            requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

        constexpr sentinel_t<Base> base() const;
    };
}

```

```

    friend constexpr bool operator==(const CI<Const>& y, const sentinel& x)

    template<bool OtherConst = !Const>
        requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst>>>
    friend constexpr bool operator==(const CI<OtherConst>& y, const sentinel& x)
};
}

```

```
constexpr explicit sentinel(sentinel_t<Base> end);
```

Effects: Initializes `end_` with `end`.

```

constexpr sentinel(sentinel_t<!Const> s)
    requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

```

Effects: Initializes `end_` with `std::move(s.end_)`.

```
constexpr sentinel_t<Base> base() const;
```

Effects: Equivalent to: `return end_;`

```

    friend constexpr bool operator==(const CI<Const>& y, const sentinel& x);

    template<bool OtherConst = !Const>
        requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst>>>
    friend constexpr bool operator==(const CI<OtherConst>& y, const sentinel& x)

```

Effects: Equivalent to: `return y.count() == 0 || y.base() == x.end_;`

§ 8.5. Feature test macro

Add the following feature test macro to [version.syn]:

```
#define __cpp_lib_lazy_counted_iterator date // also in <iterator>
```

§ 9. Opens

- `basic_const_iterator` defines `iterator_category` only if the underlying iterator models `forward_iterator`. `move_iterator` defines it even for `input_iterator` case. We followed `move_iterator` example in the wording above, but maybe `basic_const_iterator` behavior is what should be used here?
- `views::lazy_counted` and `lazy_take` effectively increment the underlying iterator when reaching `count` for cases like `random_access_iterator`, as they use tools other than `lazy_counted_iterator` in such cases (`span`, returning the iterator directly etc.). We kept it this way as:
 1. These types don't promise to never increment the underlying iterator for such cases, they only promise it implicitly for the cases that use `lazy_counted_iterator`.
 2. We don't think that difference is directly observable (especially as `lazy_counted_iterator` doesn't provide `base()`).
 3. We could've define these types to always use `lazy_counted_iterator` internally, but this limits their usability and means we can't recommend people to just use `lazy_take` by default.
- There was a suggestion to combine `operator*` definitions into a single `const` overload, but we aren't sure about this and would like to hear more about it.

§ 10. Implementation experience

The current wording has been implemented over Microsoft STL [\[MSFT-STL\]](#).

There is also a partial implementation, that is based as much as possible directly on the wording of this paper [\[YB-IMPL\]](#).

§ 11. Note about optimization

It's interesting to note that with any level of optimization enabled (including `-Og`!), gcc is able to "fix the issue" [\[CE-OPT\]](#) for the filter+take case (but not for `input_iterator`, of course). It's maybe even more interesting to see the mentioned optimization is not an optimizer bug, and when the filter will never return another number, it doesn't change the behavior [\[CE-OPT2\]](#).

§ 12. Acknowledgements

Many thanks to the Israeli NB members for their feedback and support, in particular Inbal Levi, Dvir Yitzchaki, Dan Raviv and Andrei Zissu. Thanks r/cpp Reddit users for their feedback on P2406R0 [\[reddit-cpp\]](#). Thanks SG9 members for their feedback and guidance.

Thanks Eddie Nolan and Robert Leahy for thier invaluable help and support in preparing R5 of this paper.

§ References

§ Informative References

[CE-FILTER]

filter+take problem example, Compiler Explorer. URL: <https://gcc.godbolt.org/z/9TjbdMn3d>

[CE-ISTREAM]

istream problem example, Compiler Explorer. URL: <https://gcc.godbolt.org/z/Eb8rdWYbP>

[CE-OPT]

Optimizer magic solves filter+take issue, Compiler Explorer. URL:
<https://gcc.godbolt.org/z/4dahzG8Gz>

[CE-OPT2]

Optimizer is right when filter really never returns, Compiler Explorer. URL:
<https://gcc.godbolt.org/z/PvMY8WeaT>

[D2578R0]

D2578R0 - Block eager input (non-forward) iterators from counted_iterator. URL:
<https://isocpp.org/files/papers/D2578R0.html>

[MSFT-STL]

GitHub - YehezkelShB/STL - forked from microsoft/STL - P2406R2-Option2 branch. URL:
<https://github.com/YehezkelShB/STL/tree/P2406R2-Option2>

[RANGE-V3-ISSUE57]

range-v3 - istream_range filtered with take(N) should stop reading at N. URL:
<https://github.com/ericniebler/range-v3/issues/57>

[REDDIT-CPP]

r/cpp comments on P2406R0. URL:
https://www.reddit.com/r/cpp/comments/orw4q8/wg21_july_2021_mailing/h6kqu7y

[US-46-107]

US 46-107 25.5.7.1 [counted_iterator] Too many iterator increments. URL:
<https://github.com/cplusplus/nbballot/issues/523>

[YB-IMPL]

GitHub - YehezkelShB/LazyCountedIterator. URL:

<https://github.com/YehezkelShB/LazyCountedIterator>